

Enterprise Agent Architecture & Multi-Agent Patterns

Ramesh Sannareddy

From Single Agents to Enterprise Agent Systems



Enterprise Architecture

Agent Roles & Patterns

Planner → Executor → Reviewer

State & Workflow Orchestration

Human-in-the-Loop

Hands-On: Python Build

From One Agent to an Enterprise System

A basic agent follows a simple cognitive loop — **Understand** → **Plan** → **Use Tools** → **Respond**. That's sufficient for isolated, well-scoped tasks. But enterprise environments demand far more.

A Single Agent Can

- Understand natural language requests
- Plan a sequence of actions
- Invoke tools and APIs
- Formulate a response

An Enterprise System Must Also Handle

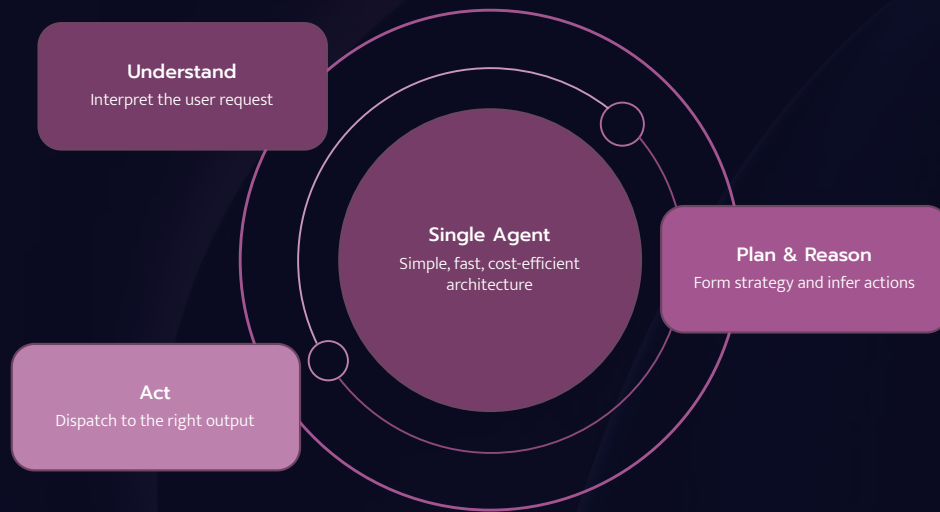
- Multiple concurrent responsibilities
- Complex, multi-step workflows
- State management and auditability
- Validation, approvals, and escalation
- Failure recovery and retry logic
- Integration with enterprise systems and APIs

❏ **The Architectural Question:** When should one agent do everything, and when should responsibilities be separated across multiple agents?



The Single-Agent Architecture

Start simple — a single agent remains the right choice for many tasks. It receives a user request, reasons through it, and dispatches to the appropriate tools. This model is fast, cost-efficient, and easy to debug.



Advantages

- **Simple architecture**
One reasoning loop, one context window, one deployment unit.
- **Lower latency & cost**
No inter-agent communication overhead or token duplication.
- **Easier debugging**
A single trace covers the full execution path.



As responsibilities grow, the single agent becomes increasingly difficult to control, evaluate, and maintain.

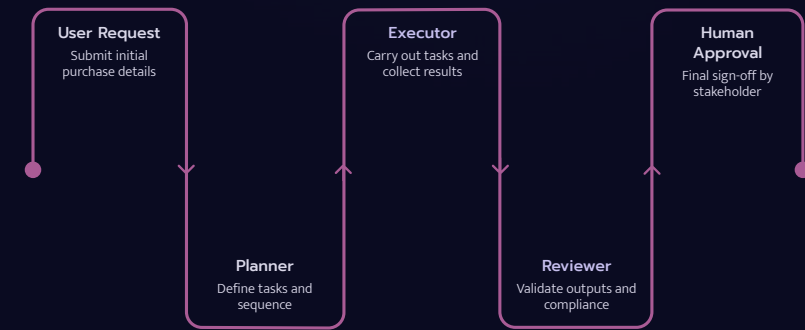
Why Decompose an Agent?

Enterprise workflows contain distinct, separable responsibilities. Bundling all of them into a single agent creates an uncontrollable monolith. Consider a **Purchase Request** workflow — it spans eight discrete phases, each with different logic and risk levels.

Purchase Request Steps

- 1 Understand request
- 2 Validate request
- 3 Check company policy
- 4 Check budget
- 5 Gather procurement info
- 6 Review proposed action
- 7 Obtain approval
- 8 Execute purchase

Decomposed Agent Workflow



Decomposition assigns each phase to an agent with a **defined scope**, making the system testable, auditable, and recoverable at each stage.

Multi-Agent Does NOT Mean Multiple Chatbots

A multi-agent system is a **structured, executable workflow** — not a collection of independently chatting LLMs. Each agent occupies a defined role with explicit inputs, outputs, tools, and handoff conditions.



Planner

Decomposes a complex goal into a structured, executable sequence of steps.



Router

Determines which downstream agent or capability should handle an incoming request.



Executor

Performs the required actions by selecting and invoking the appropriate tools.



Reviewer

Validates outputs against policy, business rules, and quality constraints.



Coordinator

Manages the overall workflow state and orchestrates inter-agent handoffs.



Key Principle: Separate responsibilities only when that separation creates clear architectural value — not by default.

Core Enterprise Agent Patterns

Six foundational patterns recur across enterprise agent systems. Understanding each pattern's purpose, triggers, and boundaries is essential before composing them into a full system.



Router Agent

Determines which specialized capability should handle an incoming request based on intent classification.



Planner Agent

Breaks a complex business goal into a structured, ordered sequence of executable steps.



Executor Agent

Performs the required actions by invoking tools, APIs, and databases with controlled inputs.



Reviewer Agent

Independently validates outputs before they proceed, catching policy violations and errors.



Coordinator

Owns workflow state and manages the lifecycle of multiple agents across a long-running task.

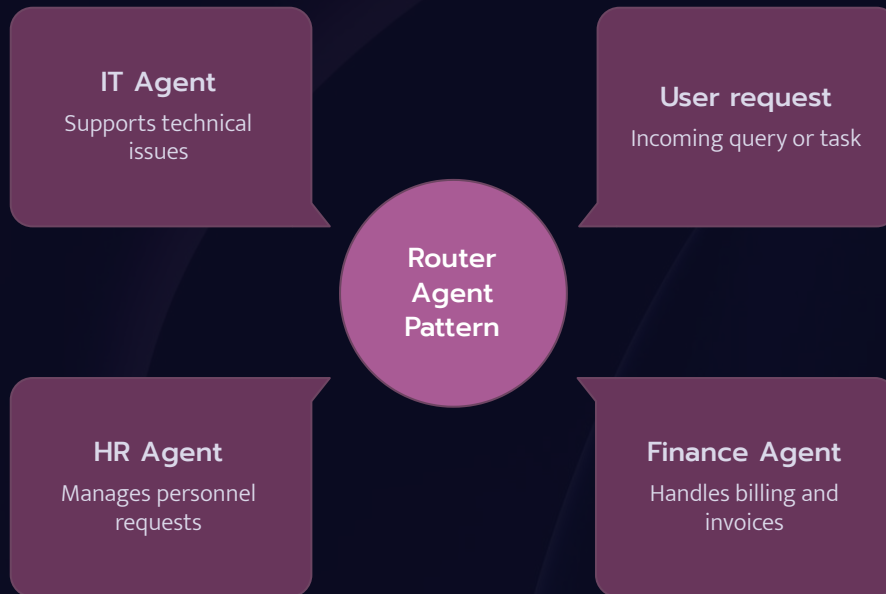


Human Approval

Transfers control to a human decision-maker at designated, high-consequence control points.

Router Agent Pattern

The Router Agent acts as the intelligent front door to a multi-agent system. It classifies incoming requests and dispatches them to the appropriate downstream agent or capability — without performing the work itself.



📌 **Design consideration:** Routing should be deterministic where possible, and fully observable when LLM-based classification is used.

Routing in Practice

HR → Leave Agent

"How many vacation days do I have remaining?"

Router identifies HR intent and routes to the Leave Management Agent.

IT → Helpdesk Agent

"My laptop is not connecting to VPN."

Router identifies IT intent and routes to the Helpdesk Agent.

Finance → Procurement Agent

"I need to order hardware for the new team."

Router identifies Finance/Procurement intent and dispatches accordingly.

Planner Agent Pattern

The Planner converts an ambiguous business goal into a **structured, executable plan**. Its output is not prose — it is a machine-readable sequence of steps that downstream executors can act upon reliably.

Example Goal: Onboard a New Employee

01

Validate employee information

02

Check equipment availability

03

Create system accounts

04

Schedule induction sessions

05

Generate onboarding checklist

06

Create welcome communication

What Makes a Good Plan

→ Structured and typed

Each step has a clear action, expected inputs, and outputs — not free-form text.

→ Independently executable

Steps can be handed to an Executor without further interpretation.

→ Auditable

The plan itself is a record — reviewable before any action is taken.

❏ The planner should produce **structured, executable plans** — not uncontrolled prose that an executor must re-interpret.

Executor Agent Pattern

The Executor is the only agent that touches external systems. It takes structured plan steps as input, selects the appropriate tools, validates inputs, and returns structured results — including failures. Scope discipline is critical.



Executor Responsibilities

- **Select the right tool**
Match each plan step to the correct API, database query, or file operation.
- **Validate inputs before acting**
Prevent malformed or unauthorized calls from reaching external systems.
- **Capture and structure results**
Return typed outputs that the Reviewer can evaluate programmatically.
- **Handle failures explicitly**
Retry transient errors, surface permanent failures — never silently swallow them.

❏ **Principle:** The Executor executes. It should not silently redesign the plan when something is harder than expected.

Reviewer Agent Pattern

The Reviewer provides an **independent validation layer** between execution and completion. It never assumes an agent's output is correct — it systematically checks policy compliance, business rules, data quality, and authorization before allowing the workflow to proceed.

What the Reviewer Checks

Policy Compliance

Does the output conform to company policy and regulatory requirements?

Required Fields

Are all mandatory data fields present and correctly populated?

Business Rules

Does the result satisfy domain-specific validation logic?

Safety & Authorization

Is the action within the agent's permitted scope and risk threshold?

Decision Outcomes

✓ APPROVE

Output meets all criteria — workflow advances to the next stage or completion.

✗ REJECT → Replan

Output fails validation — Planner is invoked to revise the approach and retry.

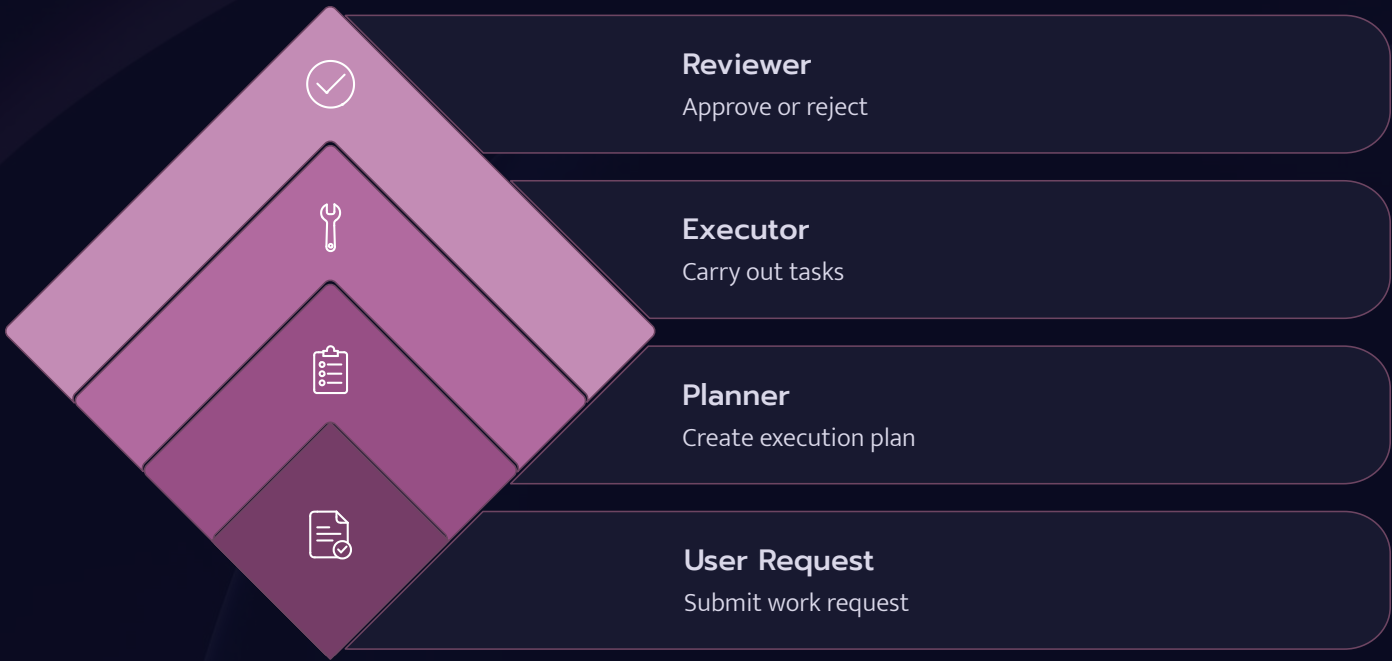
⚠ ESCALATE

Output exceeds automated approval threshold — routed to human decision-maker.

*Example: A purchase exceeding the \$10,000 approval threshold triggers **REJECT → Request Human Approval**.*

Planner → Executor → Reviewer

This three-agent sequence is the foundational pattern for enterprise agent workflows. Together they form a **controlled pipeline** — not a collection of independent conversations — where each stage has a clear contract with the next.



Planner Converts intent into a typed, executable plan. Owns task decomposition and step sequencing.	Executor Acts on plan steps using controlled tool access. Returns structured results — including failures.	Reviewer Independently validates outputs. Approves, rejects, or escalates based on defined criteria.
---	--	--

The agents form a **controlled workflow**. Separation of concerns — planning, acting, validating — is what makes the system auditable and recoverable.

Human-in-the-Loop

Autonomous does not mean uncontrolled. Enterprise systems must define explicit control boundaries where human judgment is required before consequential actions are executed. The agent automates preparation — the human controls the outcome.

When to Require Human Approval



Financial Transactions

Purchases, payments, and budget allocations above defined thresholds.



External Communications

Outbound messages to customers, partners, or regulators.



Contract Changes

Modifications to legal agreements or commitments with third parties.



Security-Sensitive Operations

Permission changes, access grants, and irreversible system modifications.

Design Principle

Automate preparation aggressively; control consequential actions deliberately.

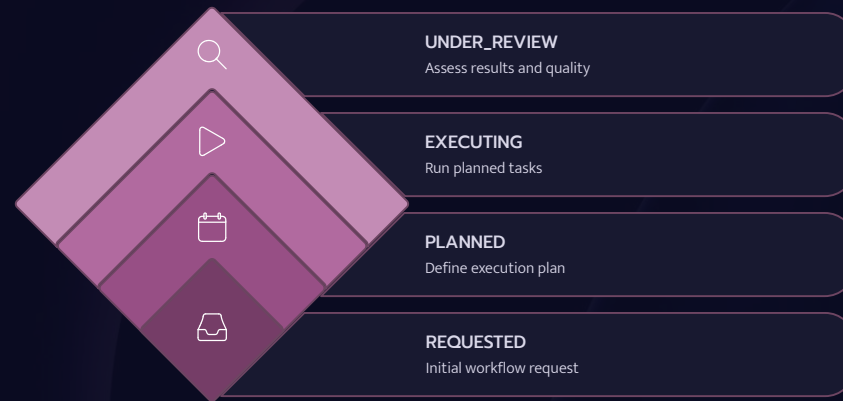
The agent does the analytical heavy lifting — validating data, checking policy, preparing the recommendation. The human reviews a complete, ready-to-approve package rather than starting from scratch.

This pattern **reduces approval friction** while maintaining **meaningful human control** at the moments that matter most.

State Management

Long-running enterprise workflows cannot rely on a single context window. Explicit state management is what allows an agent system to be **paused, resumed, debugged, audited, and recovered** — even after failures or human interruptions.

Workflow State Machine



What State Must Capture

Workflow Status

Current lifecycle stage — planned, executing, under review, completed.

Step Completion

Which steps are done, which are pending, which have failed.

Tool Results

Structured outputs from every tool call, including errors.

Approvals & Agent Outputs

Decision records for both human approvals and agent-level validations.

⚠ Without explicit state, complex agent workflows become impossible to resume, audit, or recover from failures.

Failure and Escalation

Enterprise agents operate in environments where failure is not an edge case — it is a certainty. **Failure handling is a first-class architectural concern**, not an afterthought. Every agent must be designed with explicit recovery paths.

Categories of Failure

Infrastructure Failures

Tool unavailable, API timeout, authentication failure.

Data Failures

Invalid inputs, missing required fields, malformed responses.

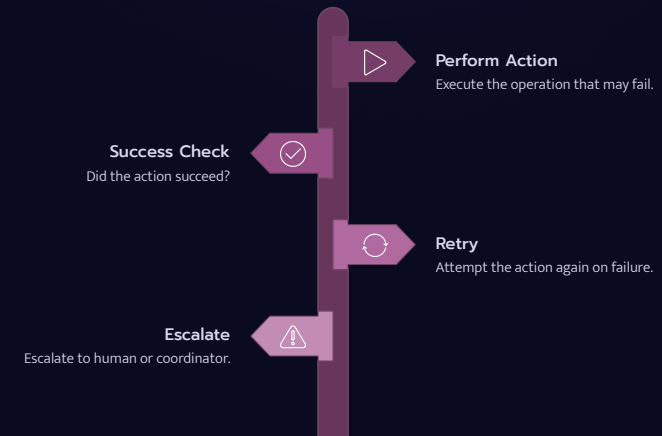
Logic Failures

LLM produces invalid output, business rule violation, policy breach.

Process Failures

Reviewer rejection, human approval timeout, escalation required.

Recovery Pattern



Retry

For transient, recoverable errors — with backoff and attempt limits.

Replan

For structural failures — the Planner revises the approach.

Escalate

For unresolvable failures — human or coordinator takes over.

When NOT to Use Multiple Agents

More agents do not automatically produce a better system. Decomposition introduces coordination overhead, latency, token cost, and debugging complexity. Apply it only when the architectural benefits justify the cost.

✓ Prefer a Single Agent When

→ Workflow is simple

The task has limited branching and fits within a single reasoning loop.

→ Responsibilities are tightly coupled

Separating them would create artificial boundaries with no benefit.

→ Latency and cost matter

Each additional agent adds round-trips, tokens, and coordination logic.

→ Debugging simplicity is required

A single trace is far easier to inspect than a distributed multi-agent log.

⚡ Consider Multiple Agents When

→ Responsibilities are clearly separable

Planning, execution, and validation have distinct logic and failure modes.

→ Independent validation is required

A separate Reviewer cannot be trusted if it shares state with the Executor.

→ Human checkpoints are mandatory

The workflow must pause for approval at defined control boundaries.

→ Different tool permissions are needed

Least-privilege access is enforced at the agent boundary level.

📌 **Architectural Rule:** Use the minimum number of agents necessary to create clear separation of responsibility.

Lab 4: Build a Multi-Agent System

Participants will implement a complete **Planner** → **Executor** → **Reviewer** workflow from scratch in Python — no frameworks, no abstractions. The goal is to understand every layer of the architecture through direct implementation.



Planner

Converts a business request into a structured, typed plan with discrete executable steps.



Executor

Selects and invokes Python tool functions.
Returns structured results including errors.



Reviewer

Validates execution results against policy rules.
Approves, rejects, or escalates.



Workflow Controller

Maintains workflow state, handles failures, and supports replanning across agent handoffs.



Human Approval

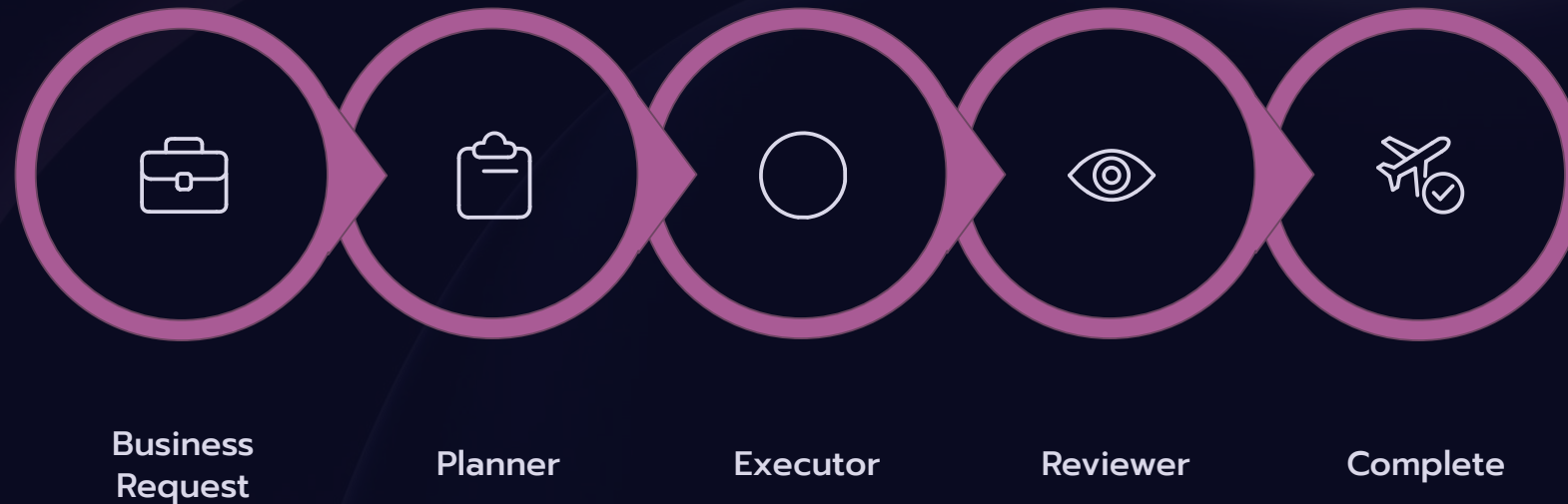
Interrupts the workflow at defined control points to collect a human decision before proceeding.



Constraint: Build the complete workflow directly in Python. No LangGraph. No CrewAI. No AutoGen.

Lab Architecture

The lab implements a complete, end-to-end multi-agent pipeline. Each component maps directly to a pattern from this session — making the architecture both a learning artifact and a reusable reference implementation.



Input

A business request string — the same interface a real enterprise user would provide.

Tool Results

Structured outputs from each tool call, captured in workflow state for the Reviewer.



Structured Plan

A typed Python object — not prose — that the Executor consumes step by step.



Final Output

A completed, approved artifact — or a clear escalation record if human action is required.



Lab Scenario: Purchase Approval Agent

"Process the purchase request for 25 laptops for the new engineering team."

Planner

- Parse and understand the purchase request
- Validate requirements and quantities
- Identify applicable procurement policy
- Determine required approval thresholds

Executor

- Query inventory availability
- Retrieve current procurement policy
- Verify budget allocation
- Prepare structured purchase request

Reviewer

- Validate policy compliance
- Check against approval threshold
- Verify all required information is present

Human → Agent

- Human approves or rejects the purchase
- Agent generates final approved purchase order

What Participants Should Observe

This lab is not just a coding exercise — it is a direct demonstration of why multi-agent architecture exists. Each design decision in the lab maps to a real enterprise engineering concern.



Planning ≠ Execution

The Planner and Executor have different reasoning tasks, different tool access, and different failure modes. Mixing them reduces control.



Controlled Tool Access

The Executor is the only agent that touches external systems — this boundary enforces least-privilege and simplifies auditing.



Independent Validation

The Reviewer cannot be trusted if it shares context with the Executor. Independence is what makes validation meaningful.



Explicit Control Boundaries

Human approval is a hard interrupt — not a soft suggestion. The workflow cannot proceed without it at designated checkpoints.



Separation of Responsibility

Multi-agent architecture is fundamentally about assigning each responsibility the right level of autonomy and the right constraints.

Architecture Decision Checklist

Before introducing multiple agents into a system, every architectural decision should be justified against this checklist. Unanswered questions indicate design gaps that will surface as production incidents.

1

Responsibility

What specific responsibility is being separated into its own agent?

2

Value

Does the separation improve reliability, control, or auditability?

3

Tools

What tools does each agent need — and should access be strictly limited?

4

Inputs & Outputs

What typed information does each agent receive and return?

5

State Ownership

Who owns workflow state? How is it persisted and recovered?

6

Failure Points

Where can failure occur, and what is the recovery strategy at each point?

7

Human Approval

Which actions require a human decision before the workflow may continue?

8

Observability

How will the workflow be logged, traced, and evaluated in production?

9

Simplicity Check

Could a well-scoped single-agent design achieve the same result with less complexity?

From Lab to Enterprise

The architecture you implement today is not a toy — it is the foundation upon which the capstone enterprise POC will be built. The course follows a deliberate progression, adding one layer of capability at a time.

1

LLMs

Foundation models and prompting fundamentals.

2

Tool Calling

Connecting models to APIs, databases, and functions.

3

Memory & State

Persisting context across multi-step interactions.

4

Multi-Agent Systems

Planner, Executor, Reviewer — this session.

5

Observability & Control

Tracing, evaluation, and human-in-the-loop patterns.

6

Enterprise AI Agent POC

A production-ready blueprint built on everything above.

Why This Progression Matters

Each layer builds on the last. Understanding how a single agent works — and where it breaks — is what makes multi-agent design decisions meaningful rather than cargo-culted.

By the time you reach the capstone, you will have implemented every component yourself: the planner logic, the tool executor, the reviewer, the state machine, and the human approval interrupt.

That foundation is what separates architects who **design enterprise agent systems** from engineers who simply **use frameworks**.

Key Takeaways

Enterprise agent architecture is a discipline of **deliberate decomposition**, explicit control, and engineered reliability — not a matter of wiring together as many agents as possible.



Start Simple

Use a single agent when the workflow is straightforward. Complexity should be justified, not assumed.



Decompose Deliberately

Separate responsibilities only when separation creates clear architectural value — reliability, control, or auditability.



Give Agents Explicit Roles

Planner, Executor, Reviewer, Router, Coordinator — each with defined scope, tools, inputs, and outputs.



Control Consequential Actions

Human approval, authorization checks, and escalation paths are not optional for enterprise-grade systems.



Make State Explicit

Track progress, results, failures, and approvals in a persistent state object — not in the LLM's context window.



Design for Failure

Every agent must have a defined strategy: retry, replan, or escalate. Failure handling is architecture.

Good agent architecture is not about using more agents. It is about giving each responsibility the right level of autonomy and control.